

MÓDULO 2: Ownership & Memory Management

Este es, sin exagerar, el módulo más importante de todo el curso. Ownership es lo que le permite a Rust dar seguridad de memoria **sin garbage collector**. Todo lo demás (borrow checker, lifetimes, smart pointers) son consecuencias directas de este sistema.

2.1 El Problema que Ownership Resuelve

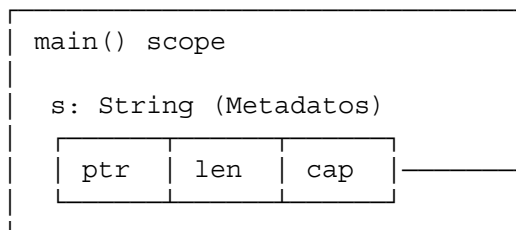
Antes de las reglas, entiende el problema:

- **C/C++:** tú gestionas memoria manualmente (malloc/free, new/delete). Riesgo: use-after-free, double-free, memory leaks.
- **Java/Python/Go:** un Garbage Collector gestiona la memoria por ti. Seguro, pero con costo en runtime (pausas, overhead).
- **Rust:** el compilador rastrea, en tiempo de compilación, quién es responsable de liberar cada dato. Cero costo en runtime, cero bugs de memoria (en código seguro).

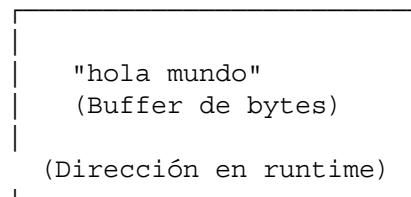
2.2 Diagrama Mental: Stack vs. Heap

Diagrama Mental: Distribución de Memoria String

STACK (LIFO, estático, alto rendimiento)



HEAP (Dinámico, datos variables)



Cuando escribes `String::from("hola")`, en el **stack** vive una estructura de 3 campos (puntero, longitud, capacidad), y en el **heap** vive el contenido real de los bytes. Esto es la clave de todo lo que sigue.

2.3 Las Reglas de Ownership

1. Cada valor en Rust tiene una variable que es su "owner" (dueño).
2. Solo puede haber UN owner a la vez.
3. Cuando el owner sale de scope, el valor se libera automáticamente (Drop).

```
fn main() {
    {
        let s = String::from("hola"); // s es owner, memoria
        reservada en heap
        println!("{}", s);
    } // <- aquí s sale de scope, Rust llama automáticamente a
    `drop(s)`, libera el heap
    // println!("{}", s); // ❌ no compilaría: s ya no existe aquí
}
```

Esto es **RAII** (Resource Acquisition Is Initialization), igual que en C++, pero enforced por el compilador, no por convención.

2.3.b Trait Drop Explícito, RAII y Trazabilidad de Recursos

El patrón **RAII** (*Resource Acquisition Is Initialization*) garantiza que cualquier recurso (memoria heap, file descriptors, locks, sockets) se libere de forma determinista tan pronto como la variable dueña abandone su *scope*.

En Rust, esto se implementa mediante el trait `std::ops::Drop`. La función `std::mem::drop(x)` permite ceder el ownership de una variable y destruirla antes de que finalice su bloque léxico, forzando la ejecución inmediata de su destructor.

Diagrama Mental: Ciclo de Vida y Ejecución de Drop

```
Inicio del bloque léxico {
    let recurso = Recurso::abrir(); // [1] Adquisición (Heap, descriptor o lock)
    // Operaciones con el dato...
    drop(recurso);                  // [2] Liberación manual: ejecuta
Drop::drop(&mut self)              //      -> El recurso se destruye YA, sin
esperar al final.
    // Lógica subsiguiente...      // [3] `recurso` invalidado: su uso causaría un
error de tipos.
} // Finalización del scope
```

Código Compilable

```
struct SocketConexion {
    id: u32,
    direccion: String,
}

impl SocketConexion {
    fn abrir(id: u32, direccion: &str) -> Self {
        println!("[ABRIR] Conexión {} establecida con {}", id,
direccion);
        Self {
            id,
            direccion: direccion.to_string(),
        }
    }
}

impl Drop for SocketConexion {
    fn drop(&mut self) {
        // Lógica de limpieza determinista
        println!("[DROP] Cerrando socket {} ({})) y liberando
descriptor", self.id, self.direccion);
    }
}
```

```
fn main() {
    println!("--- Inicio de main ---");
    {
        let socket1 = SocketConexion::abrir(1, "192.168.1.10:8080");
        let socket2 = SocketConexion::abrir(2, "192.168.1.20:8080");

        // Liberación explícita temprana de socket1
        println!("Liberando socket1 antes de tiempo...");
        drop(socket1);

        println!("socket1 ya fue destruido; socket2 sigue vivo");
    } // <-- socket2 sale de scope aquí y se ejecuta su Drop
    automáticamente

    println!("--- Fin de main ---");
}
```

⚠ Error típico del compilador: Llamar a .drop() directamente como método

```
fn main() {
    let mut s = SocketConexion::abrir(1, "127.0.0.1");
    s.drop(); // ❌ Llamada directa explícita al método del trait
}
```

```
error[E0040]: explicit use of destructor method
--> src/main.rs:3:7
   |
3  |     s.drop();
   |     ^^^^^ explicit destructor calls not allowed
help: consider using `drop` function
   |
3  -     s.drop();
3  +     drop(s);
   |
```

Por qué ocurre: Rust prohíbe invocar el método `s.drop()` directamente para evitar un doble destructor (*double-free*), ya que el compilador llamará a `Drop::drop` automáticamente al finalizar el scope. Para destruir un valor antes, la función libre `drop(s)` toma la propiedad (*ownership*) del valor y lo destruye de manera segura.

2.4 Move Semantics (el corazón del sistema)

```
fn main() {
    let s1 = String::from("hola");
    let s2 = s1; // ! esto NO copia el string, MUEVE el ownership
    println!("{}", s2); // ✅ funciona
    // println!("{}", s1); // ❌ ERROR: s1 ya no es válido
}
```

Diagrama del Move

ANTES del move: DESPUÉS del move (s1 = s2):
s1: [ptr|len|cap] → heap: "hola" s1: [ptr|len|cap] ← INVALIDADO por Rust
s2: [ptr|len|cap] → heap: "hola"

Por qué Rust hace esto: si permitiera que s1 y s2 apuntaran al mismo heap y ambos salieran de scope, tendrías un **double-free** (ambos intentarían liberar la misma memoria). En vez de eso, Rust invalida s1 en tiempo de compilación. Cero costo en runtime, cero bug posible.

Error típico del compilador #1: Use after move

```
fn main() {  
    let s1 = String::from("hola");  
    let s2 = s1;  
    println!("{}", {}, s1, s2); //   
}
```

error[E0382]: borrow of moved value: `s1`
--> src/main.rs:4:23
2 | let s1 = String::from("hola");
 | -- move occurs because `s1` has type `String`, which
does not implement the `Copy` trait
3 | let s2 = s1;
 | -- value moved here
4 | println!("{}", {}, s1, s2);
 | ^^ value borrowed here after move

help: consider cloning the value if the performance cost is
acceptable
3 | let s2 = s1.clone();
 | +++++++

Lectura clave: el error menciona el trait Copy. Esto nos lleva directo al siguiente concepto.

2.5 Copy vs. Move: ¿Por qué los i32 no se mueven?

```
fn main() {  
    let x = 5;  
    let y = x; // esto SÍ copia, no mueve  
    println!("{}", {}, x, y); //  ambos siguen siendo válidos  
}
```

Regla: los tipos que implementan el trait Copy (enteros, floats, bool, char, y tuplas de tipos Copy) se copian automáticamente porque:

1. Viven enteramente en el stack.
2. Copiarlos es barato (unos pocos bytes).
3. No tienen recursos en el heap que gestionar → no hay riesgo de double-free.

String, Vec, Box, etc. NO implementan Copy porque gestionan memoria en el heap — copiarlos "gratis" sería peligroso (double-free), y copiarlos "de verdad" sería costoso, así que Rust te obliga a ser explícito con .clone().

```
fn main() {
    let s1 = String::from("hola");
    let s2 = s1.clone(); // copia profunda EXPLÍCITA del heap
    println!("{}", s1, s2); // ✅ ambos válidos, cada uno con su
    propio heap
}
```

2.5.b Copy vs Clone a Nivel de Memoria

1. **Copy:** Realiza una copia bit a bit (memcpy) exclusivamente dentro del **Stack**. No requiere asignaciones en runtime ni llamadas al sistema operativo. Solo es válido para tipos cuyo tamaño es conocido en compile-time y que no implementan el trait Drop.
2. **Clone:** Es un duplicado explícito (a menudo *deep copy*) que puede involucrar al asignador de memoria (*heap allocator*), clonando buffers, descriptores o estructuras complejas.

Diagrama Mental: Stack memcpy vs Asignación en Heap

```
TRAIT COPY (Stack Only):
let a: i32 = 42;
let b = a;           [Stack: a = 42] —memcpy→ [Stack: b = 42]
(Costo: ~0 ns)
```

```
TRAIT CLONE (Heap Allocation):
let s1 = String::from("datos");
let s2 = s1.clone();
[Stack: s1] —ptr→ [Heap: "datos"]
[Stack: s2] —ptr→ [Heap: "datos" (NUEVO BLOQUE RESERVADO EN
RUNTIME)]
```

Código Compilable

```
#[derive(Debug, Clone, Copy)]
struct Coordenada {
    x: f64,
    y: f64,
} // Implementa Copy: solo vive en el Stack

#[derive(Debug, Clone)]
struct RegistroSensor {
    id: u32,
    datos: Vec<f64>, // Vec maneja memoria en el Heap -> NO puede ser Copy
}

fn main() {
    // Demostración de Copy
    let c1 = Coordenada { x: 10.5, y: 20.0 };
    let c2 = c1; // Copia implícita bit a bit
    println!("c1: {:?}, c2: {:?}", c1, c2); // c1 sigue siendo válido

    // Demostración de Clone
    let r1 = RegistroSensor {
        id: 1,
        datos: vec![0.1, 0.5, 0.9],
    };
    let r2 = r1.clone(); // Duplicación explícita del buffer en Heap
    println!("r1: {:?}, r2: {:?}", r1, r2);
}
```

2.6 Ownership y Funciones

Pasar un valor a una función también mueve (o copia) el ownership:

```
fn toma_ownership(s: String) {
    println!("Dentro de la función: {}", s);
} // s sale de scope aquí -> se libera

fn toma_copia(x: i32) {
    println!("Copia: {}", x);
} // x es Copy, no importa

fn main() {
    let s1 = String::from("hola");
    toma_ownership(s1);
    // println!("{}", s1); // ❌ s1 ya fue movido dentro de la
función

    let x = 5;
    toma_copia(x);
    println!("{}", x); // ✅ x sigue siendo válido, era Copy
}
```

Devolver valores también transfiere ownership de vuelta:

```
fn genera_string() -> String {
    let s = String::from("generado");
    s // se retorna, ownership pasa al llamador
}

fn main() {
    let mi_string = genera_string(); // mi_string es ahora el owner
    println!("{}", mi_string);
}
```

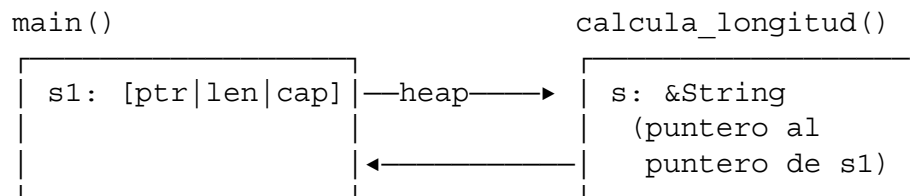
Esto sería tedioso si siempre tuvieras que devolver el valor para "recuperarlo". La solución: **Borrowing**.

2.7 Borrowing: Referencias sin Transferir Ownership

```
fn calcula_longitud(s: &String) -> usize { // &String = referencia,
"presta" el valor
    s.len()
} // s sale de scope, pero como NO es owner, no se libera nada

fn main() {
    let s1 = String::from("hola mundo");
    let len = calcula_longitud(&s1); // pasamos una referencia con &
    println!("La longitud de '{}' es {}", s1, len); // ✅ s1 sigue
siendo válido
}
```

🧠 Diagrama mental de Borrowing



s es una **referencia**: un puntero que apunta a **s1**, pero no es el owner. Cuando **s** sale de scope, no se libera nada porque no tiene ownership.

2.8 Las Reglas del Borrow Checker

1. En un momento dado, puedes tener:
 - CUALQUIER número de referencias INMUTABLES (&T), 0
 - EXACTAMENTE UNA referencia MUTABLE (&mut T)
 - pero nunca ambas al mismo tiempo.
2. Las referencias deben ser siempre válidas (no dangling references).

```
fn main() {
    let mut s = String::from("hola");
    let r1 = &s; // ✅ referencia inmutable
    let r2 = &s; // ✅ otra referencia inmutable, está permitido
    println!("{}", r1, r2);

    // r1 y r2 ya no se usan después de esta línea(NLL: Non-Lexical Lifetimes)
    let r3 = &mut s; // ✅ ahora sí, referencia mutable(r1,r2 ya "murieron")
    r3.push_str(" mundo");
    println!("{}", r3);
}
```

⚠ Error típico del compilador #2: Mezclar referencia mutable e immutable

```
fn main() {  
    let mut s = String::from("hola");  
    let r1 = &s;  
    let r2 = &mut s; // ✖  
    println!("{}", r1, r2);  
}
```

```
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as immutable
```

```
--> src/main.rs:4:14  
|  
3 |     let r1 = &s;  
|     -- immutable borrow occurs here  
4 |     let r2 = &mut s;  
|     ^^^^^^ mutable borrow occurs here  
5 |     println!("{}", r1, r2);  
|     -- immutable borrow later used here
```

Por qué esta regla existe: previene **data races** en tiempo de compilación. Si tuvieras una referencia mutable y una immutable simultáneas, alguien podría leer datos mientras otro los modifica — la fuente #1 de bugs en sistemas concurrentes en otros lenguajes. Rust lo hace **imposible de compilar**, incluso en código de un solo thread. (Este mismo mecanismo es la base de por qué Rust es "fearless concurrency" — lo veremos en el Módulo 8).

⚠ Error típico del compilador #3: Dangling reference

```
fn dangling() -> &String { // ✖  
    let s = String::from("hola");  
    &s  
} // s se libera aquí, pero devolvimos una referencia a s!
```

```
fn main() {  
    let ref_invalida = dangling();  
}
```

```
error[E0106]: missing lifetime specifier
```

```
--> src/main.rs:1:17  
|  
1 | fn dangling() -> &String {  
|                   ^ expected named lifetime parameter  
|  
= help: this function's return type contains a borrowed value,  
but there is no value for it to be borrowed from
```

Esto es oro puro: en C++, este código compilaría y crashearía (o peor, funcionaría "a veces") en runtime — es exactamente el bug de "use after free" / dangling pointer. En Rust, **es imposible que esto compile**. El compilador detecta que `s` va a morir y que devolver una referencia a algo que va a morir no tiene sentido. La solución correcta es devolver el `String` directamente (transferir ownership):

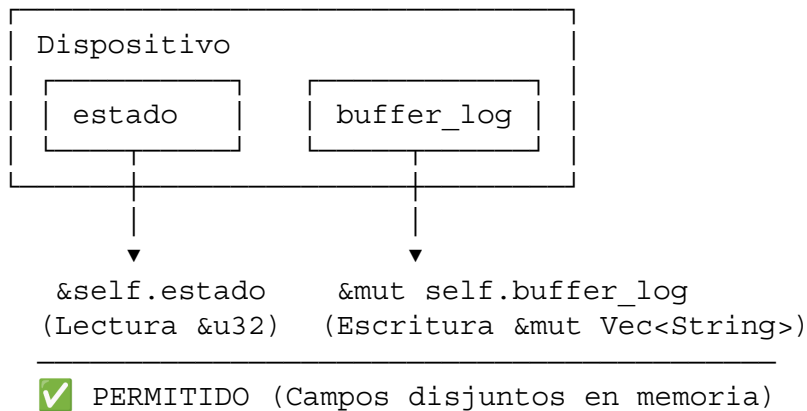
```
fn no_dangling() -> String {  
    let s = String::from("hola");  
    s // se mueve el ownership hacia afuera, todo bien  
}
```

2.8.b Borrow Splitting: Préstamos Independientes de Campos

El Borrow Checker de Rust es capaz de analizar el acceso a campos individuales de una estructura. Si accedes a `struct.campo_a` y `struct.campo_b` de forma separada, puedes tener simultáneamente una referencia inmutable a un campo y una mutable a otro, siempre que el acceso sea directo y no a través de un método que tome `&mut self` completo.

Diagrama Mental: Borrow Splitting

Struct en Memoria:



Código Compilable

```
struct Servidor {
    puerto: u16,
    conexiones_activas: Vec<String>,
}

impl Servidor {
    fn procesar_conexion(&mut self, cliente: String) {
        // Borrow splitting directo en campos disjuntos:
        let puerto_ref = &self.puerto; // Préstamo inmutable de `puerto`
        let conexiones_mut = &mut self.conexiones_activas; // Préstamo mutable de `conexiones_activas`

        conexiones_mut.push(cliente);
        println!("Registrado en el puerto: {}", puerto_ref);
    }
}

fn main() {
    let mut srv = Servidor {
        puerto: 8080,
        conexiones_activas: Vec::new(),
    };
    srv.procesar_conexion("192.168.1.50".to_string());
}
```

⚠ Error típico del compilador: Invalidar el split usando métodos auxiliares que toman &mut self

```
struct Servidor {
    puerto: u16,
    conexiones: Vec<String>,
}
impl Servidor {
    fn obtener_puerto(&self) -> &u16 {
        &self.puerto
    }
    fn registrar(&mut self, cliente: String) {
        let p = self.obtener_puerto(); // Préstamo inmutable de TODO `self`
        self.conexiones.push(cliente); // ✗ Intento de préstamo mutable
de TODO `self`
        println!("Puerto: {}", p);
    }
}

error[E0502]: cannot borrow `self.conexiones` as mutable because it is also
borrowed as immutable
--> src/main.rs:13:9
12 |         let p = self.obtener_puerto();
    |         ----- immutable borrow occurs here
13 |         self.conexiones.push(cliente);
    |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ mutable borrow occurs here
14 |         println!("Puerto: {}", p);
    |         - immutable borrow later used here
```

Por qué ocurre: Las firmas de los métodos ocultan los campos individuales tras &self o &mut self. Al llamar a self.obtener_puerto(), se toma prestada la estructura completa, bloqueando cualquier mutación subsiguiente en self.conexiones hasta que p deje de usarse.

2.8.c Antipatrones del Borrow Checker y Scopes Reducidos

Un antipatrón común consiste en extender la vida útil de una referencia mutable más allá de lo necesario, bloqueando accesos de lectura o préstamos concurrentes.

1. Extracción y finalización temprana con bloques de scope léxico {}

```
fn main() {
    let mut tabla = vec!["A".to_string(), "B".to_string()];

    // Scope reducido para aislar la mutación
    {
        let primer_elemento = &mut tabla[0];
        primer_elemento.push_str(" (Modificado)");
    } // <-- El préstamo mutable finaliza aquí

    // Ahora es completamente legal un préstamo inmutable
    println!("Tabla completa: {:?}", tabla);
}
```

2. Uso de métodos que consumen o mutan in-place sin retener referencias intermedias

```
fn procesar_cadena(texto: &mut String) {
    if let Some(posicion) = texto.find(":") {
        // En lugar de retener slices y mutar a la vez, separamos la
lógica
        let nuevo_texto = texto[..posicion].to_uppercase();
        texto.clear();
        texto.push_str(&nuevo_texto);
    }
}

fn main() {
    let mut valor = String::from("clave:valor");
    procesar_cadena(&mut valor);
    println!("Resultado: {}", valor); // CLAVE
}
```

2.9 Slices: Referencias a Partes de una Colección

```
fn primera_palabra(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i]; // slice: referencia a una porción, sin
copiar
        }
    }
    &s[..] // toda la cadena si no hay espacios
}

fn main() {
    let frase = String::from("hola mundo rust");
    let palabra = primera_palabra(&frase);
    println!("Primera palabra: {}", palabra);

    let arr = [1, 2, 3, 4, 5];
    let slice_arr = &arr[1..3]; // &[i32], slice de array
    println!("{:?}", slice_arr); // [2, 3]
}
```

 **Nota importante:** `&str` (string slice) es en realidad casi siempre lo que quieres usar como parámetro de función en vez de `&String`, porque `&str` acepta tanto `&String` como literales de texto ("hola" es directamente un `&str`). Regla práctica: **usa `&str` en parámetros, usa `String` cuando necesites ser dueño y poder modificar/hacer crecer el texto.**

2.10 Tabla Resumen: Cuándo usar qué

Situación	Usa
Necesitas ser dueño del dato, y modificarlo/hacerlo crecer	String, Vec<T> (valor propio)
Solo necesitas leer un dato que vive en otro lado	&T (referencia inmutable)
Necesitas modificar un dato que vive en otro lado	&mut T (referencia mutable)
Necesitas una copia independiente y estás dispuesto al costo	.clone()
Parámetro de función que recibe texto	&str (no &String)
Parámetro de función que recibe una colección para solo leer	&[T] (slice, no &Vec<T>)

Ejercicio Integrador — Módulo 2

Proyecto: Gestor de Inventario (sin colecciones complejas todavía)

Crea cargo new gestor_inventario e implementa:

1. Una función fn crear_producto(nombre: String, precio: f64) -> String que devuelva un String formateado tipo "Producto: {nombre}, Precio: {precio}". Debe **mover** el nombre recibido (no usar referencia aquí a propósito, para que sientas el move).
2. Una función fn aplicar_descuento(precio: &f64, porcentaje: f64) -> f64 que tome el precio **por referencia** (no tiene sentido mover un f64, pero hazlo así para practicar sintaxis de referencias — en el ejercicio de repaso verás cuándo es innecesario porque f64 es Copy).
3. Una función fn actualizar_stock(stock: &mut i32, cantidad: i32) que sume cantidad a stock usando una referencia mutable.
4. En main():
 - o Crea un producto, imprímelo.
 - o Intenta usar la variable nombre original DESPUÉS de haberla pasado a crear_producto — observa el error de compilador, léelo completo, y coméntalo en tu código explicando por qué ocurre.
 - o Aplica un descuento sin perder ownership del precio original.
 - o Actualiza el stock de un producto en una función y verifica que el cambio persiste después de la llamada.
 - o Escribe una función fn primer_producto_caro(productos: &[f64], limite: f64) -> Option<&f64> que reciba un **slice** y devuelva la primera referencia a un precio mayor que limite (usa un for con if, sin Option avanzado todavía — lo formalizamos en el Módulo 3, aquí solo practica la sintaxis del slice).

Reto extra (opcional): provoca deliberadamente un error de "cannot borrow as mutable because also borrowed as immutable" y pégamelo si no logras entender por qué ocurre — es la mejor forma de internalizar el borrow checker: romperlo a propósito y leer el mensaje.

Ejercicio 2 Integrador

Proyecto: Pool de Conexiones Manual con Trait Drop y Borrow Splitting

Crea un proyecto (cargo new pool_recursos) e implementa:

- **Estructura de Recurso:**
 - Define struct RecursoConexion { id: u32, en_uso: bool }.
 - Implementa Drop para RecursoConexion, imprimiendo un log de advertencia si la conexión se destruye mientras en_uso == true.
- **Estructura del Pool:**
 - Define struct Pool { recursos: Vec<RecursoConexion>, log_eventos: Vec<String> }.
- **Manejo de Borrow Splitting:**
 - Implementa un método fn adquirir_y_auditar(&mut self, indice: usize) que:
 - Obtenga una referencia mutable al recurso en self.recursos[indice].
 - Registre el evento insertando una entrada en self.log_eventos.
 - Realiza ambas operaciones en el mismo cuerpo de función sin provocar colisiones en el Borrow Checker mediante acceso directo a los campos.
- **Liberación con Scopes:**
 - En main(), crea un pool, adquiere una conexión dentro de un bloque léxico reducido {} y utiliza drop() manual para simular la liberación temprana del pool antes del cierre del programa.